MicroPython Programming Introduction

1. MicroPython Introduction

MicroPython is a Python language implementation specially developed for embedded devices such as the micro:bit. It is developed and maintained by an international team of volunteers and is free software. MicroPython supports basic data types (strings, integers, floating-point numbers, booleans), data structures (lists, dictionaries, sets), classes, exception handling, generators, list comprehensions, and other features.

MicroPython can run directly on the micro:bit without the need for a compiler.

2. How to Program with MicroPython

The Micro:bit official documentation recommends two MicroPython programming methods: the offline Mu Editor and the online browser editor. The online editor is only suitable for basic micro:bit cases and is not recommended for the current Lastbit car project.

3. Offline Editor

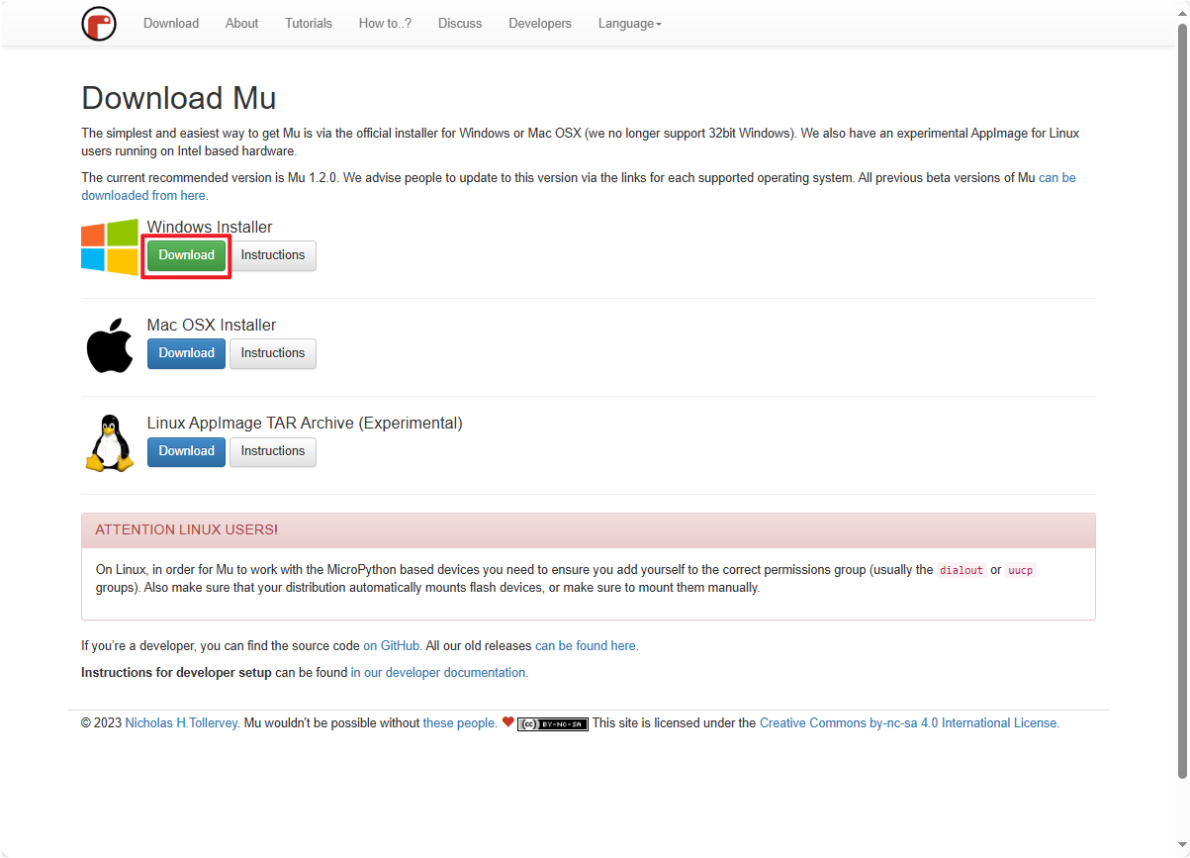
3.1 Software Download and Installation

As another MicroPython programming method for beginners, Mu Editor is also widely loved because it does not require a network connection and runs completely locally.

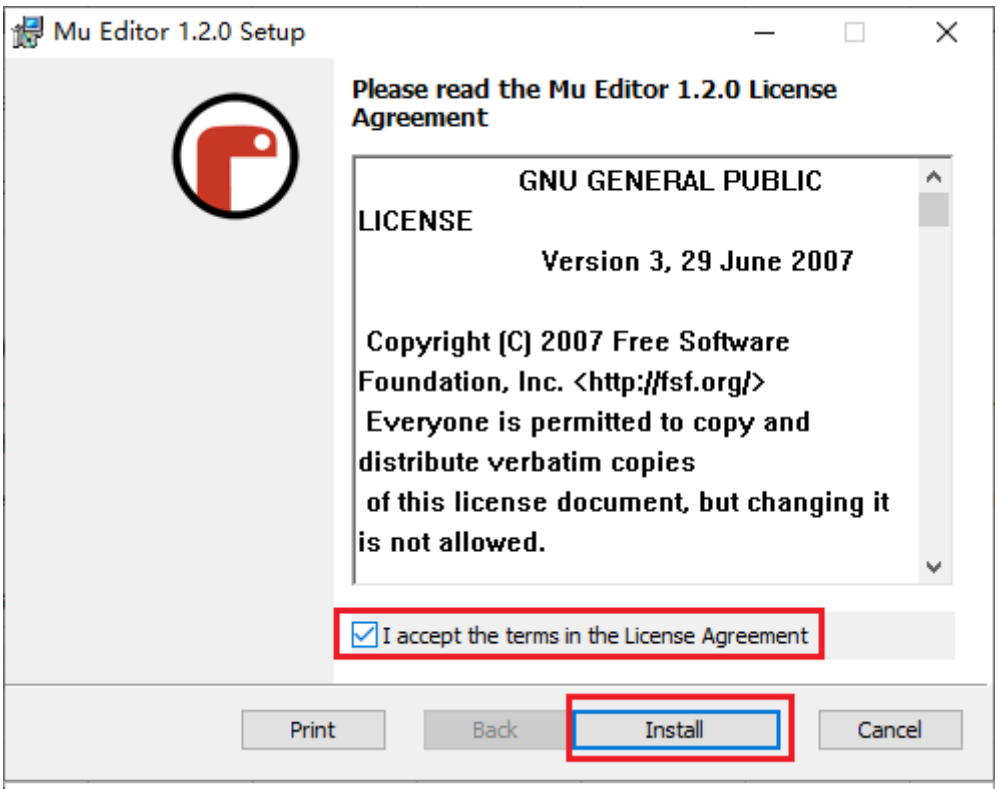
We provide a Mu Editor installation package for 64-bit systems that you can download yourself, or you can download it from the official website.

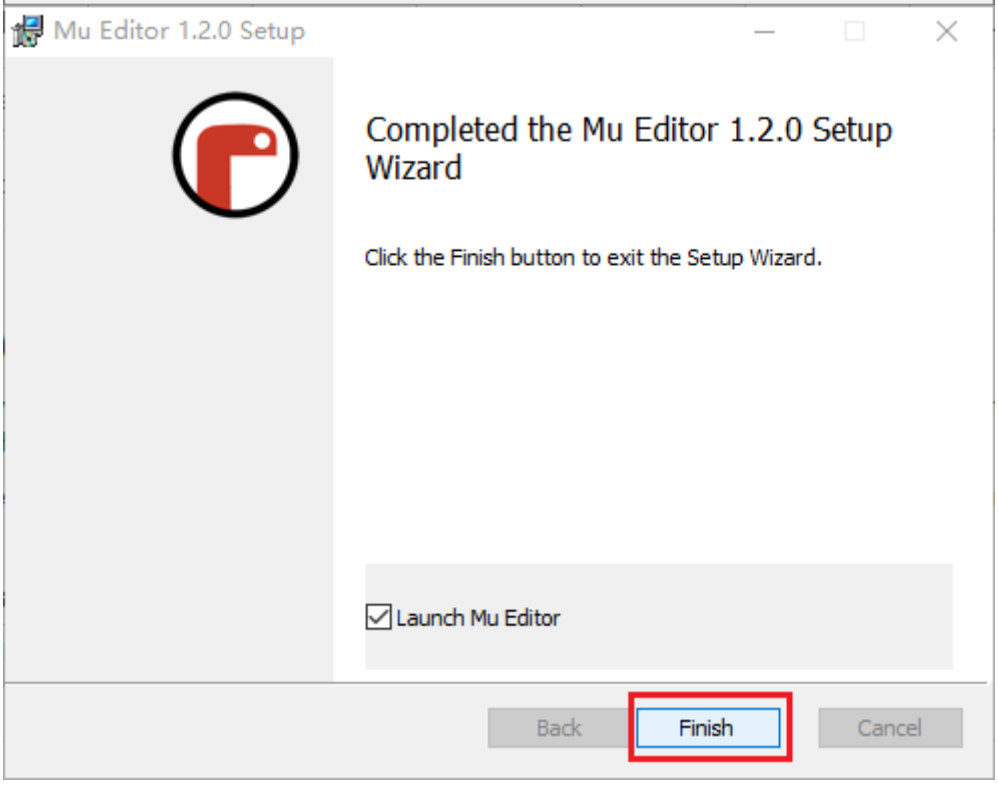
Download Mu

1. Next, it will jump to the software version selection page. Select your current computer system to download and install the software. Here, choose the Windows version to download according to your computer system.



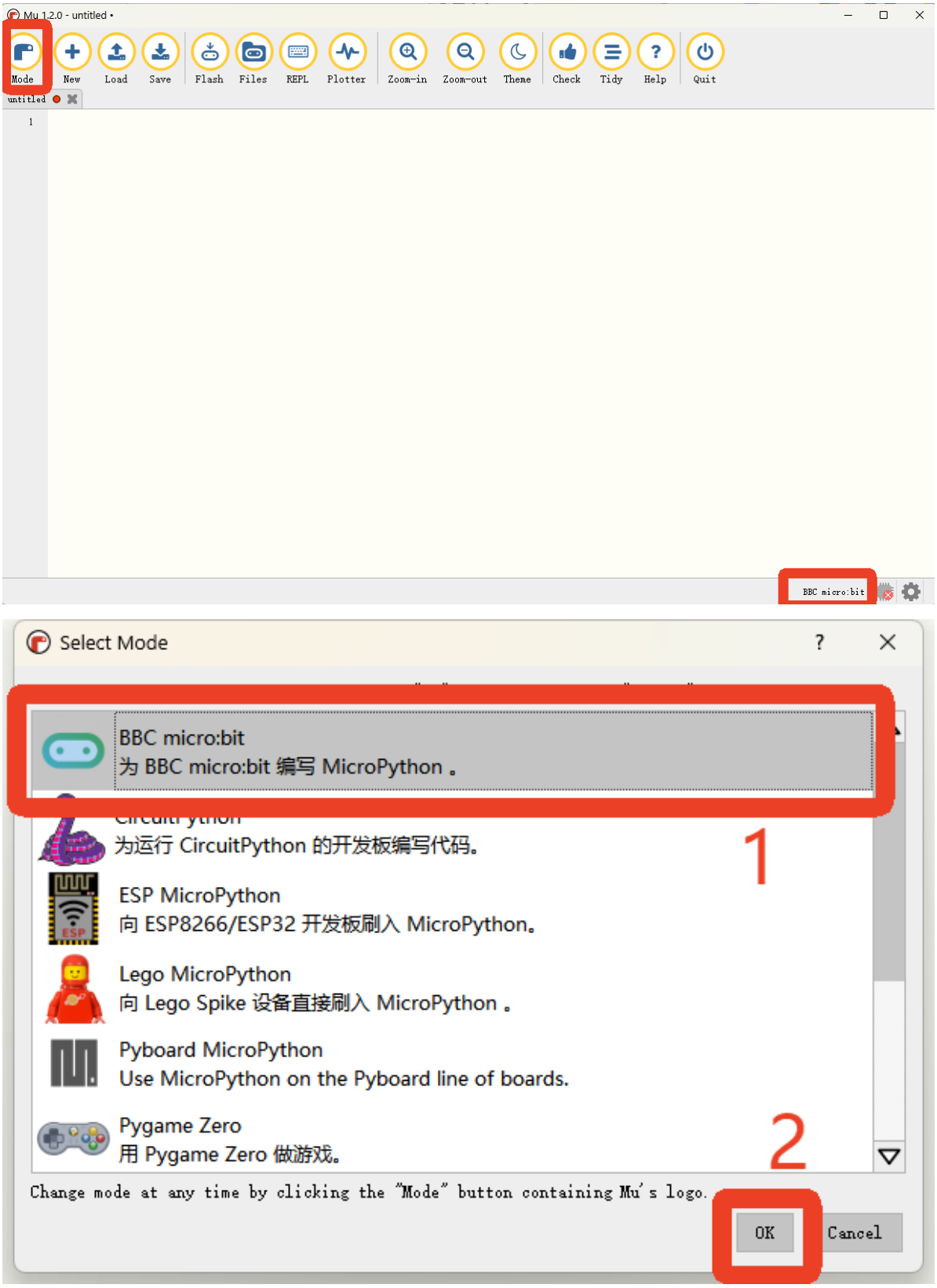
2. After the download is complete, double-click to install. After checking the box to agree to the agreement, click Install and wait for the installation to complete.



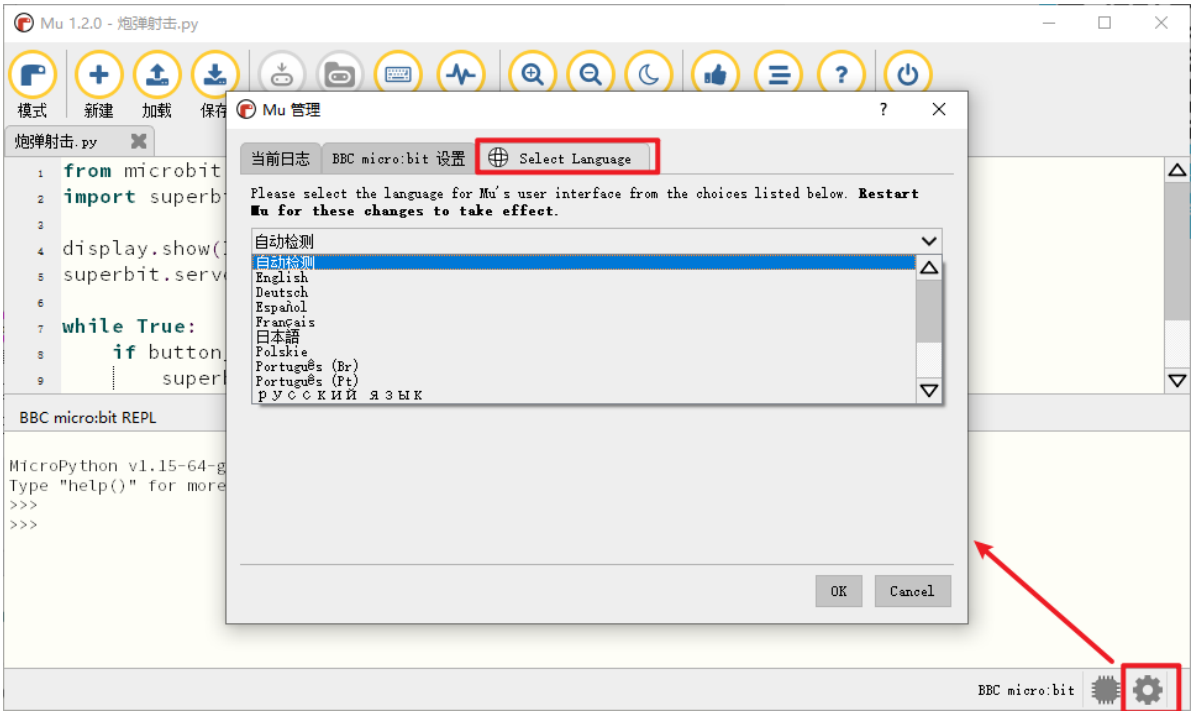


3. After the program is installed, an MU shortcut will appear on the computer desktop 

. Double-click this shortcut to run Mu. There should be a Microbit icon in the bottom right corner of Mu, which is the normal status. If it shows other content, click [Mode] in the top left corner and change it to BBC micro:bit.

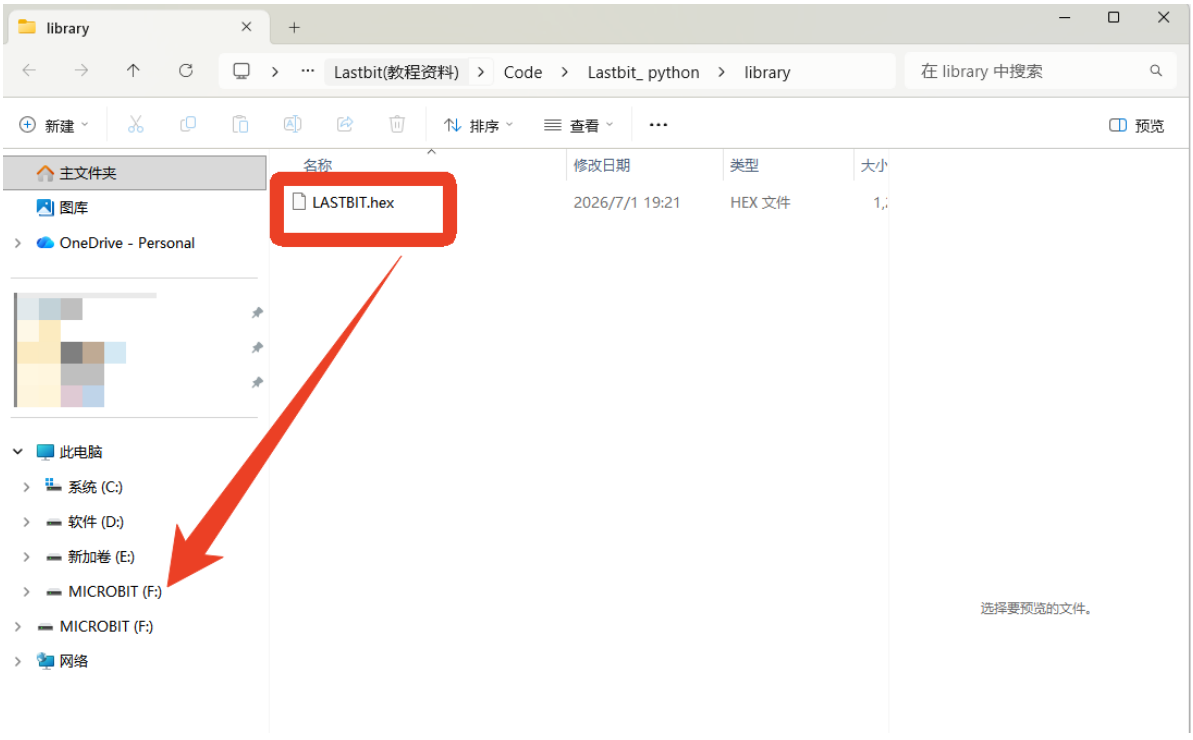


You can select the language in the settings icon at the bottom right corner.



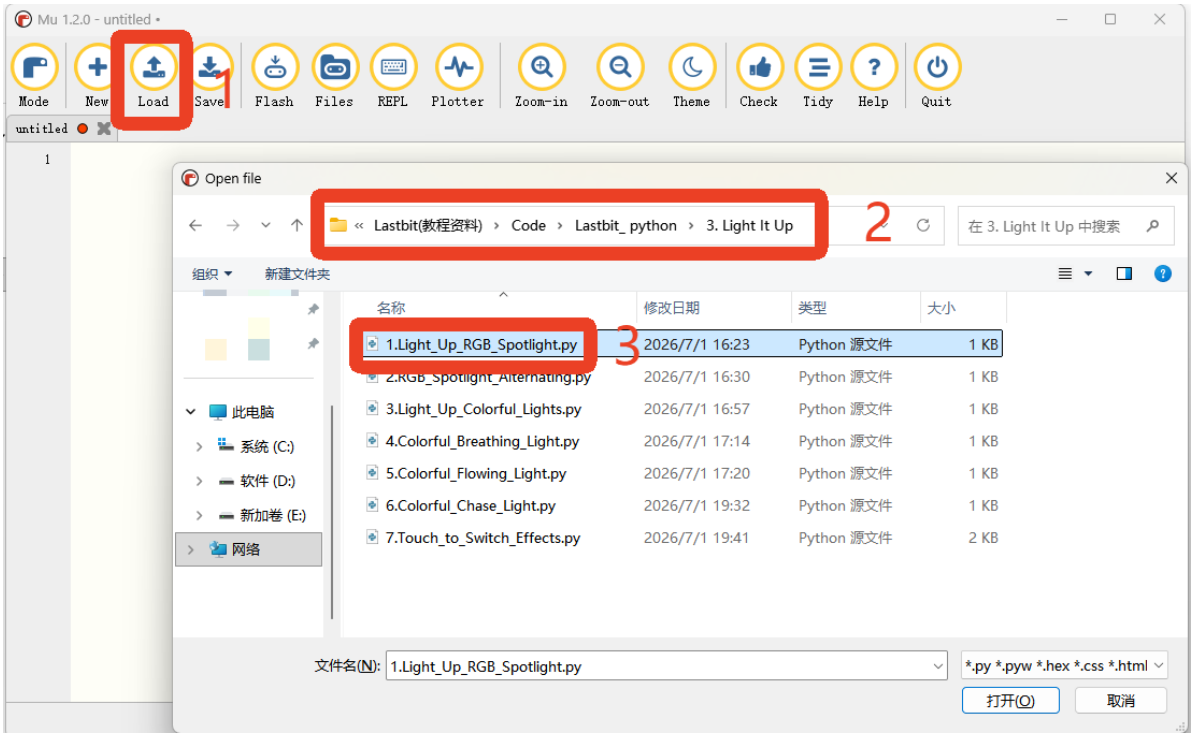
3.2 Preliminary Preparation: Flash the Lastbit Library File

Before MicroPython programming, you must first copy the provided `LASTBIT.hex` to the corresponding drive letter of the Micro:bit. In the following example, the drive letter is F, but please refer to the actual drive letter recognized by the system.

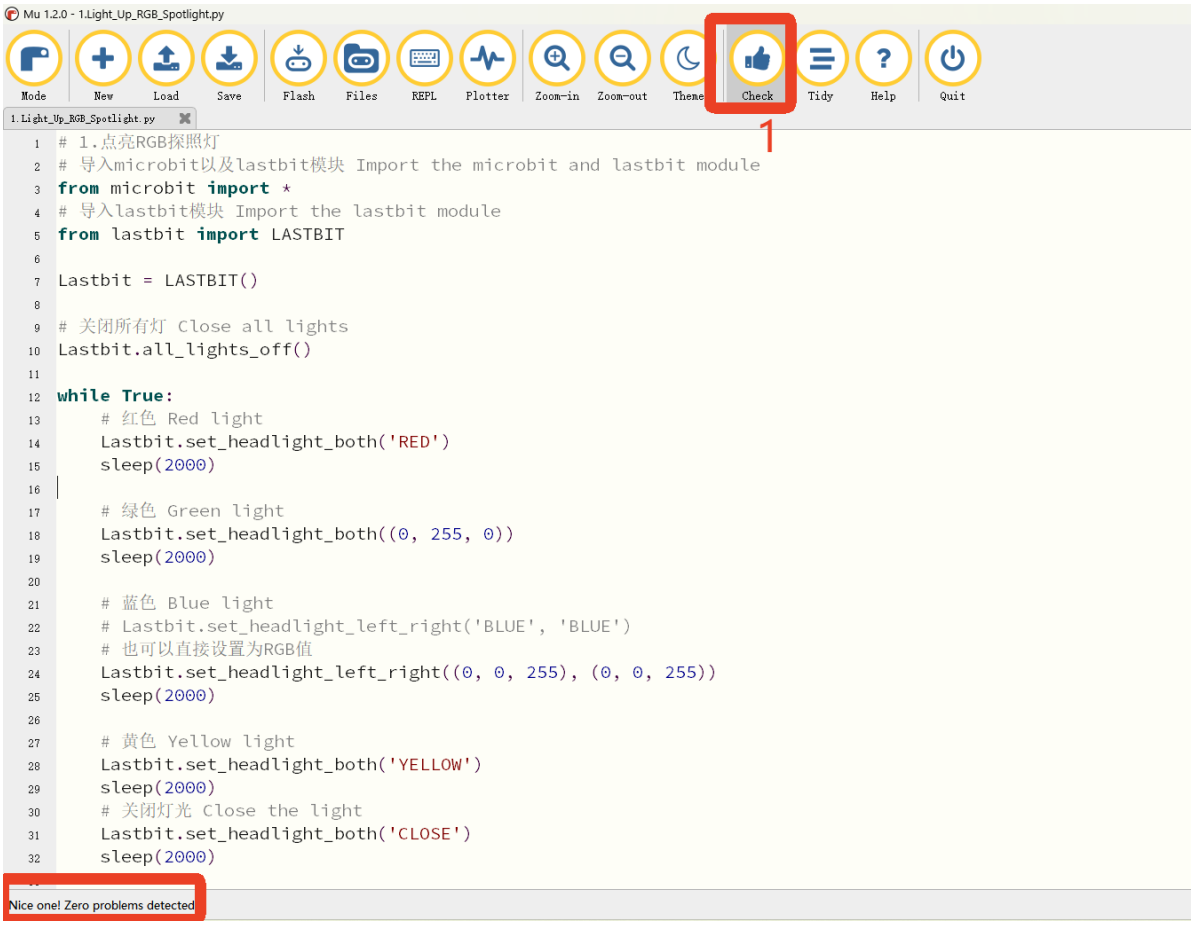


3.3 Import the Example Source Code

1. For convenient testing, you can click "Load" to import the example source code we provide. This example uses "1.Light_Up_RGB_Searchlight" as an example.

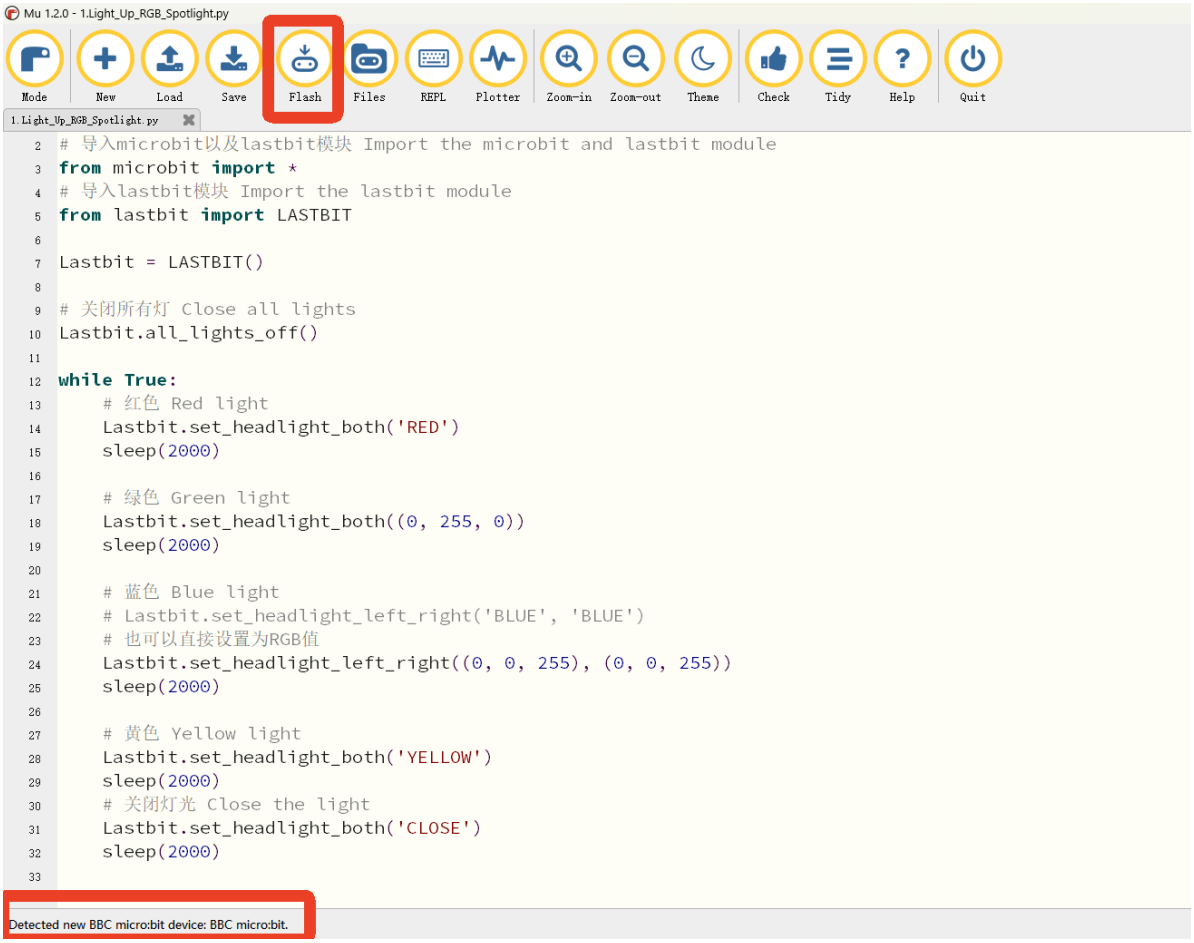


2. After the import is complete, click "Check" to confirm that the syntax is correct.

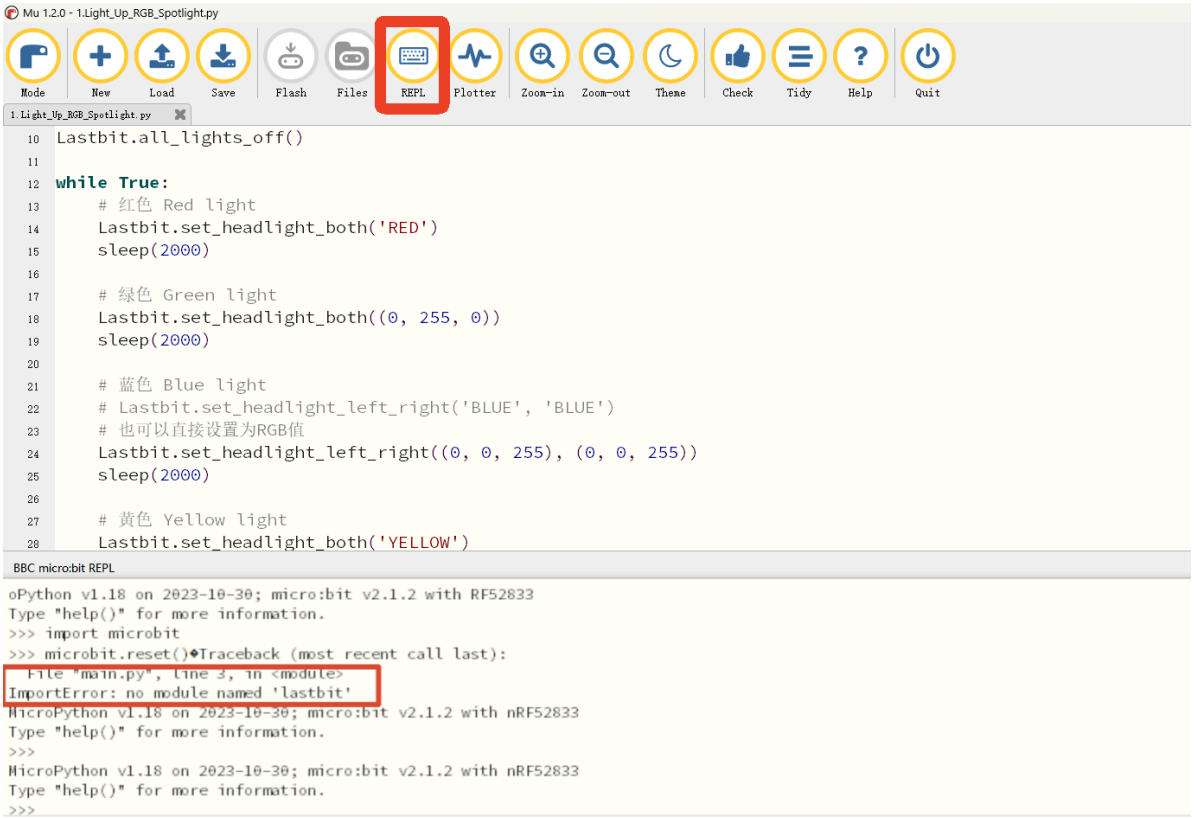


3.4 Flash the Code

1. Use a MicroUSB cable to connect the Micro:bit to the computer, and the program will prompt that a device has been detected. After a short while, click "Flash" to write the program to the Micro:bit.



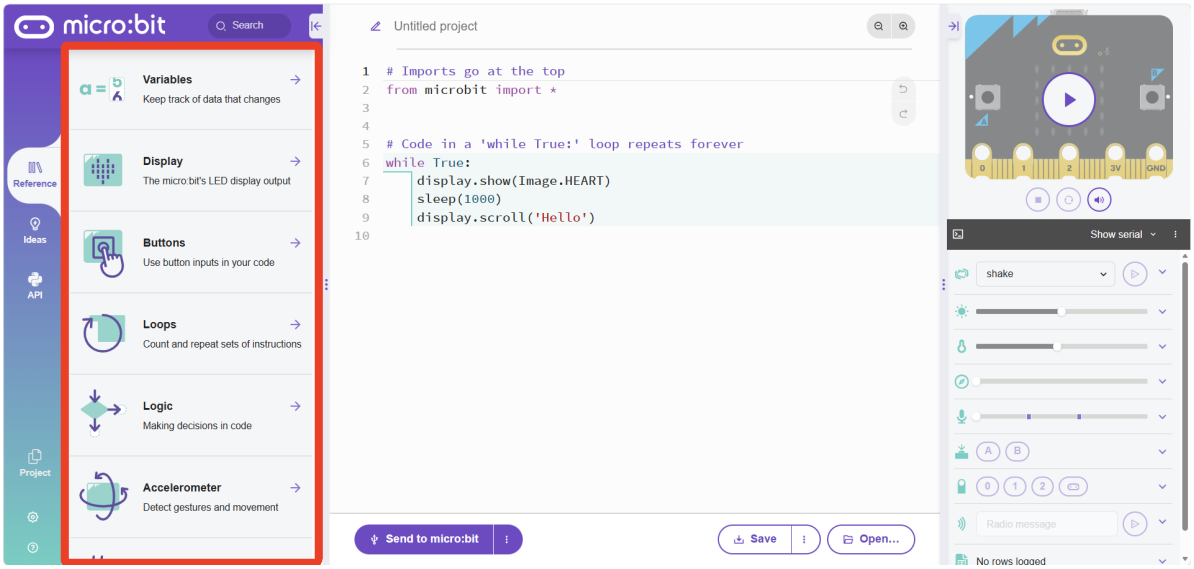
2. After flashing is complete, the lights will cycle through red, green, blue, yellow, and off in sequence. If no error occurs, it means the car lights are working normally.
3. If the switch is turned on after flashing and scrolling letters appear on the LED matrix, please troubleshoot according to the following steps.
4. Click "REPL". If `ImportError: no module named 'lastbit'` appears, it means that `LASTBIT.hex` was not flashed correctly.



5. If this error occurs, the "Flash" button may become unselectable. In this case, click "REPL" again and you can continue with the next flash.
6. If the problem still exists, please rewrite the provided `LASTBIT.hex` to the corresponding drive letter of the Micro:bit, and then try flashing the MicroPython program again.

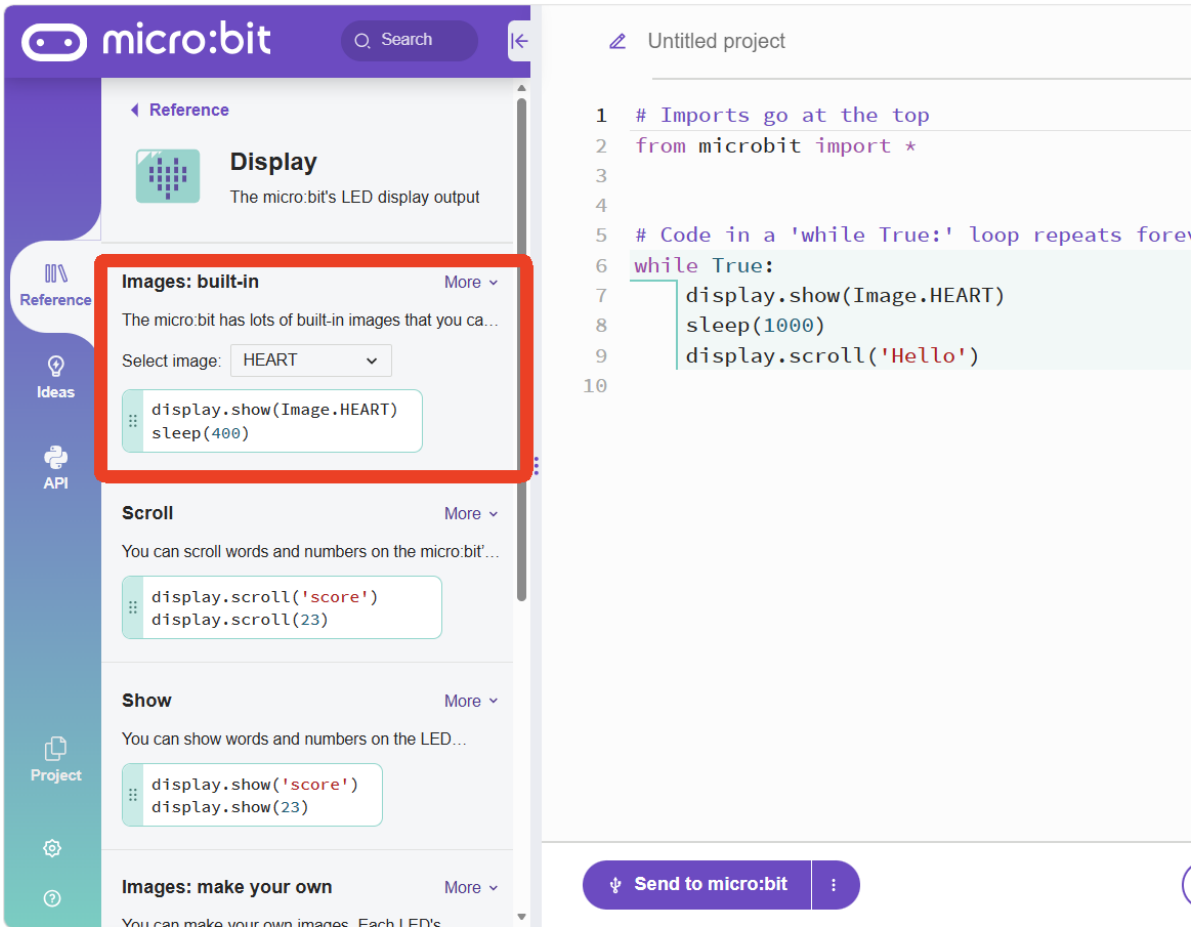
4. Online Editor (Skip for Lastbit Car)

[micro:bit Python Editor]

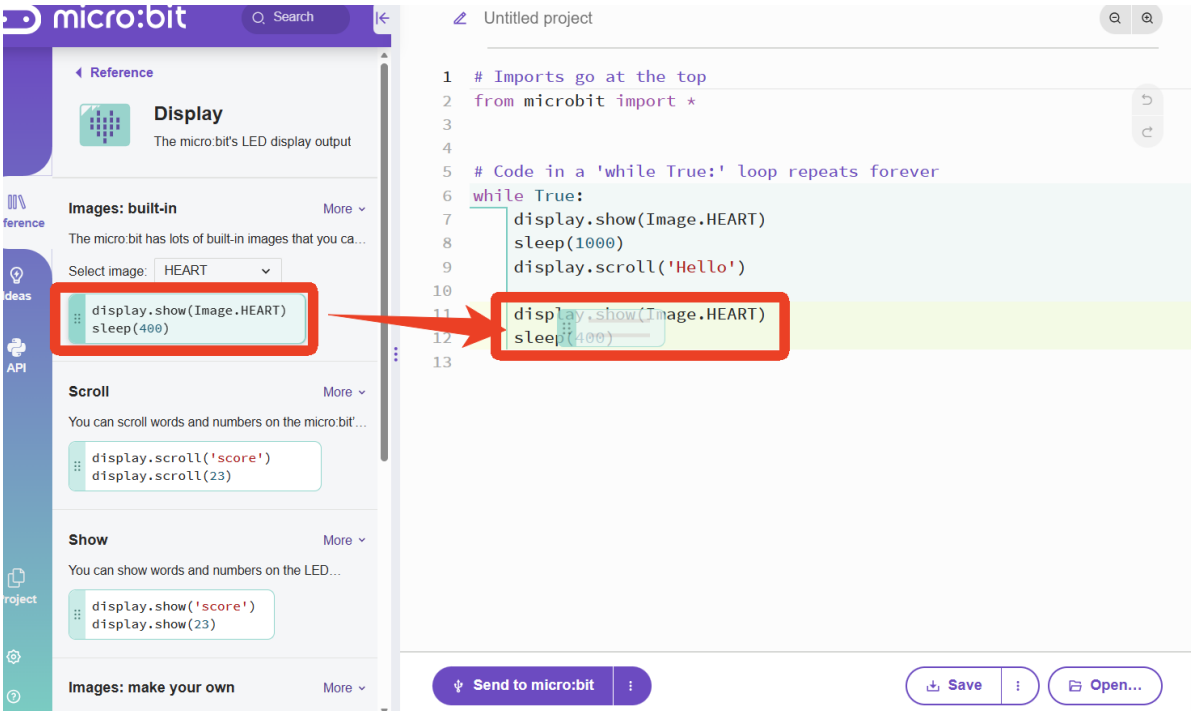


Similar to MakeCode, the left side is the input categories, the middle part is the code editing area, and the right side is the simulation simulator.

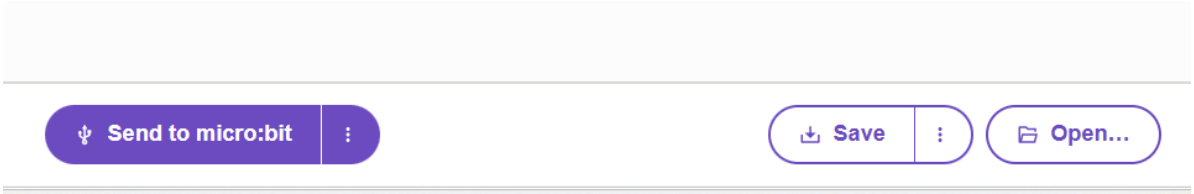
The official website provides example code under different categories, with comments included.



The code blocks support drag and drop. You can directly drag them into the code editing area in the middle, and automatic indentation is supported.

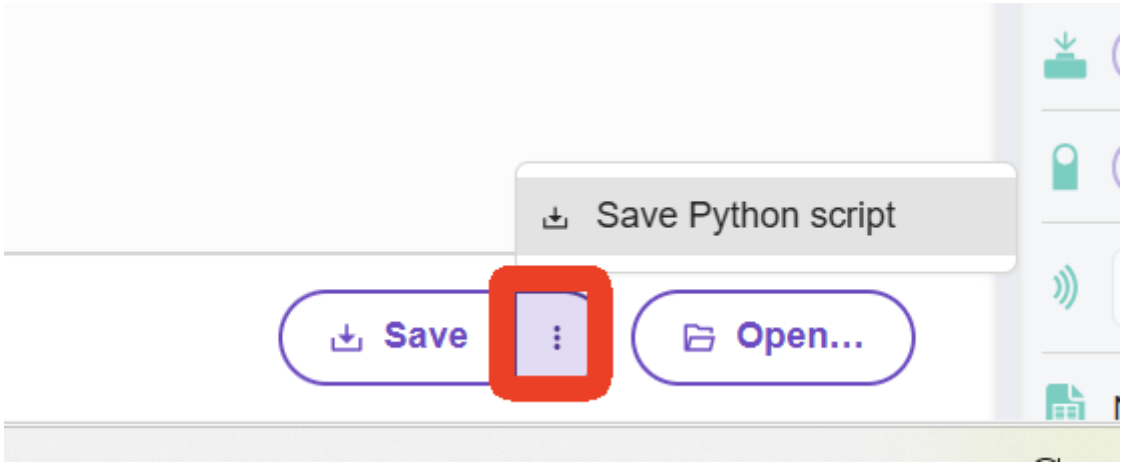


The save and download functions in the lower area are similar to MakeCode.



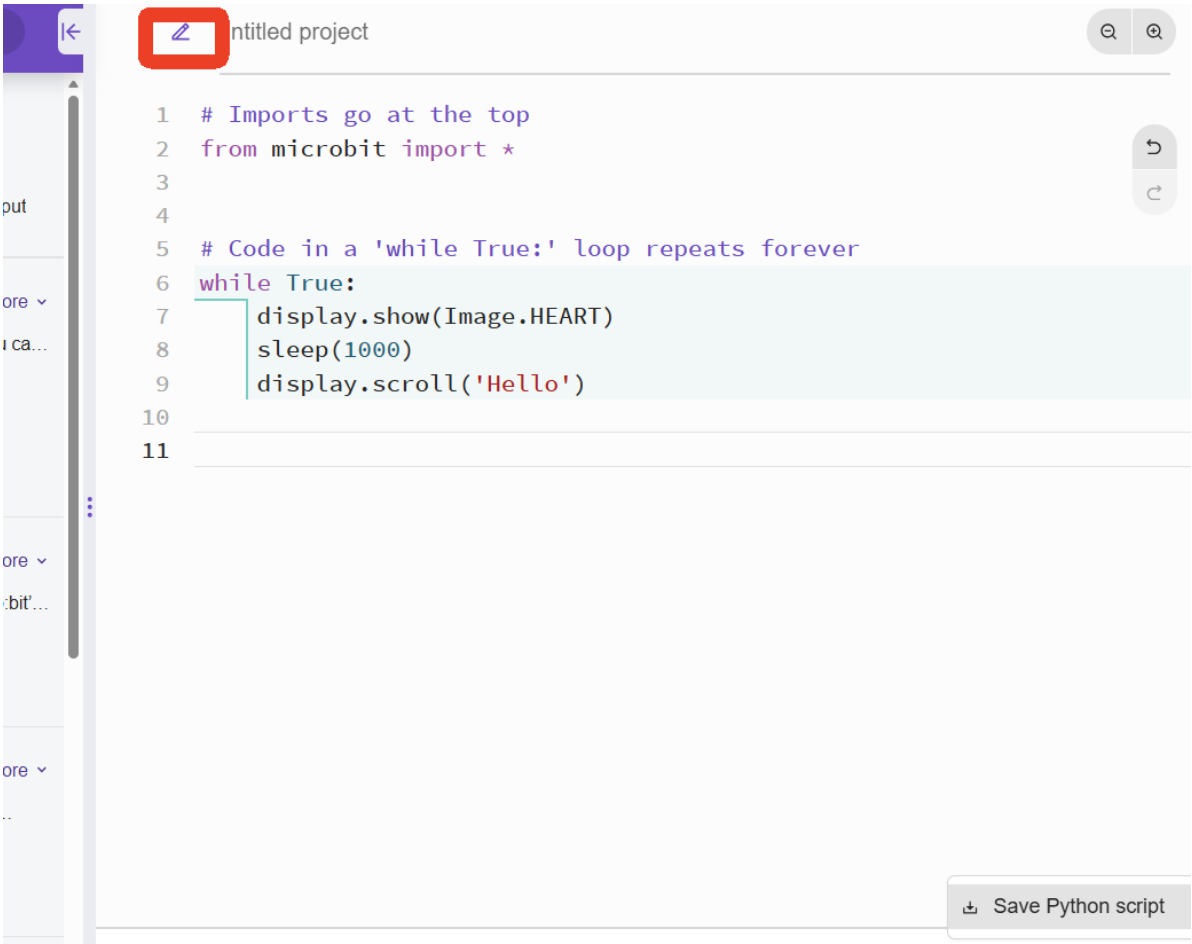
`Send to micro:bit` will prompt you to connect the microbit. After pairing, the current code will be automatically flashed to the Microbit.

`Save` will save the code in the current code editing area as hex to your local computer, and it also supports saving in the MicroPython source file format.

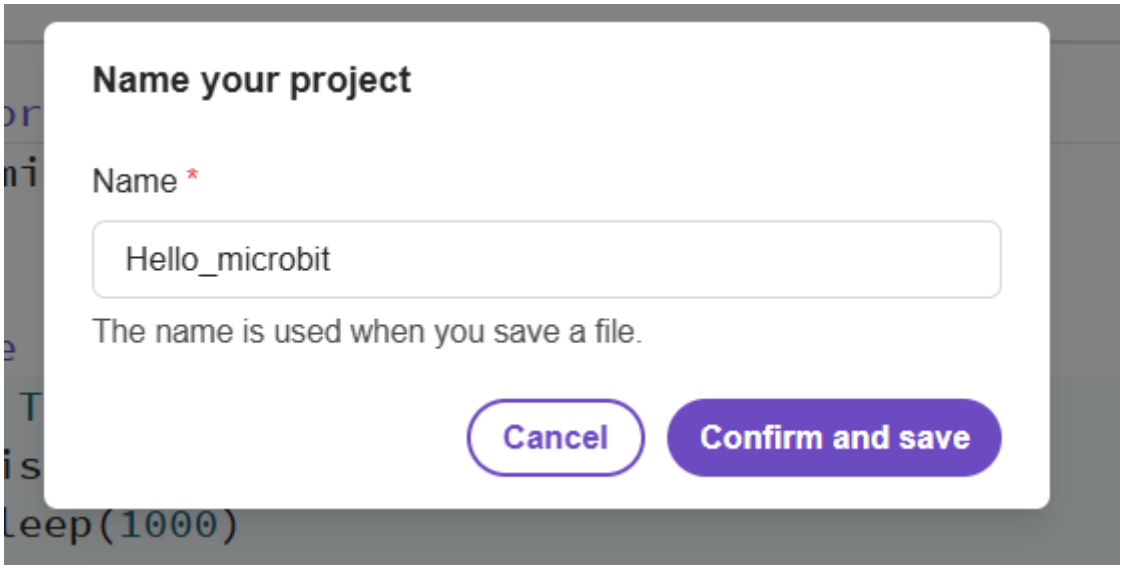


`Open` can load the saved MicroPython source program into the editor.

If you want to name your project, you can click the pencil icon above the editing area.



The editor will prompt you to enter the project name. Here, "Hello_microbit" is used as an example. Click Confirm, and every time you click download, the name will become Hello_microbit.hex.



We have introduced the common features of the online editor here. If you are interested, you can explore them on your own.